# Claude Code CLI — Team Training

## BazaarX · React + Spring Boot + PostgreSQL · 120 Minutes

> **One project. Every feature. No live demos — all examples show expected output.**
> Everything runs on a single laptop. All you need is Claude Code installed.

---

## Before You Start — 10 min

```bash
# Prerequisites: Node.js 18+
npm install -g @anthropic-ai/claude-code

claude --version   # verify
claude login       # one-time auth
```

**The project we reference throughout:**

```
bazaarx/
├── frontend/          React + TypeScript + Tailwind
├── backend/           Spring Boot 3 · Java 21 · JPA
├── db/migrations/     Flyway SQL
└── .claude/           ← everything we build today lives here
    ├── CLAUDE.md
    ├── settings.json
    ├── commands/
    ├── skills/
    └── agents/
```

---

---

## 1 · CLAUDE.md — Give Claude a Brain · 15 min

**The problem:** Every new session, Claude knows nothing about your project. You re-explain the stack. Every. Single. Time.

**The fix:** One file at the project root. Loaded automatically. Forever.

---

### Create `CLAUDE.md`

```markdown
```

```
# BazaarX

## Stack
- Frontend: React 18, Vite, TypeScript, TailwindCSS, React Query
- Backend: Spring Boot 3, Java 21, Maven, Spring Data JPA, JWT auth
- Database: PostgreSQL 16, Flyway migrations
- Payments: Stripe (INR, amounts in paise)

## Conventions
- All DB access via JPA Repositories — no native SQL in services
- Controllers use DTOs — never return entities directly
- API base path: `/api/v1/`
- Frontend calls backend only through `src/api/` Axios clients
- Prices are integers in **paise** (₹1 = 100 paise). Divide by 100 to display.

## Package Structure
- `controller/` — REST endpoints, no business logic
- `service/`    — all business logic, @Transactional on writes
- `repository/` — JPA only, extends JpaRepository
- `entity/`     — @Data @Builder Lombok entities
- `dto/`        — request/response with Bean Validation

## Never
- No business logic in controllers or repositories
- Never DROP TABLE without explicit confirmation
- Never commit application-secrets.properties
```

---

### What this looks like in practice

You ask Claude without giving any context:

```
You: How are prices stored in BazaarX?
```

```
Claude: Prices are stored as integers in paise — ₹1 = 100 paise.
        Divide by 100 before displaying to the user.
        This applies to price_paise, total_paise, and unit_price_paise columns.
```

Claude read it from the file. You typed nothing extra.

---

```
You: I need to add a discount feature. What files need to change?
```

```
Claude: Based on BazaarX conventions, you'll need:

  backend/
  ├── entity/Discount.java          — @Entity @Data @Builder
  ├── repository/DiscountRepository  — extends JpaRepository
  ├── dto/DiscountRequest.java       — @NotNull @Min validation
  ├── dto/DiscountResponse.java
  ├── service/DiscountService.java   — @Transactional on writes
  └── controller/DiscountController  — @RestController /api/v1/discounts

  frontend/
  └── src/api/discountApi.ts         — Axios client

  db/migrations/
  └── V5__add_discount_table.sql
```

> 💡 **The wow:** Claude produced a correct, convention-aligned plan from a single sentence. No stack explanation. No hand-holding.

---

### Rules for a good CLAUDE.md

| ✅ Put in | ❌ Leave out |
|---|---|
| Stack, conventions, table names | Long prose explanations |
| "Never do X" rules | Tutorials or how-tos |
| Short bullet points | Content that belongs in Skills |

> Every line loads on every session start. Short = fast + cheap.

---

### Two levels

```
~/.claude/CLAUDE.md       global — your personal preferences across all projects
bazaarx/CLAUDE.md         project — committed, every teammate gets it
```

---

---

## 2 · Settings & Permissions — Guardrails for the Team · 10 min

**The problem:** Without guardrails, Claude can run any shell command. A badly-worded prompt

could delete files or corrupt data.

**The fix:** A committed JSON file that controls exactly what Claude can and can't do.

---

### Create `.claude/settings.json`

```json
{
  "model": "claude-sonnet-4-6",
  "allowedTools": ["Read", "Write", "Bash", "Task", "WebSearch"],
  "denyTools": ["Bash(rm -rf*)", "Bash(drop table*)", "Bash(truncate*)"],
  "env": {
    "NODE_ENV": "development",
    "SPRING_PROFILES_ACTIVE": "local"
  }
}
```

---

### The hierarchy

```
~/.claude/settings.json            User-level  (personal, not committed)
bazaarx/.claude/settings.json      Project     (committed, whole team)
Enterprise policy                  Org-wide    (overrides everything)
```

Commit `.claude/settings.json`. Every teammate gets the same guardrails on `git pull`.

---

### What each setting does

| Setting | Purpose |
|---|---|
| `allowedTools` | Whitelist — only these tools can run |
| `denyTools` | Pattern-block dangerous commands |
| `model` | Default model for the whole project |
| `env` | Inject env vars into every Claude session |

---

### What a blocked command looks like

Claude tries to run a denied command:

```
Claude: I'll run the cleanup script...

[BLOCKED] Bash(rm -rf build/) matched denyTools pattern "Bash(rm -rf*)".
This tool call was not executed.
```

The block happens before execution. No damage. No override possible via prompt.

---

---

## 3 · Slash Commands — Stop Repeating Yourself · 15 min

**The problem:** You type the same 10-line prompt every time you do a PR review, or every time you add a Spring Boot feature. That's pure waste.

**The fix:** Save it once as a `.md` file. Tab-autocomplete. One keystroke.

---

### How they work

```
bazaarx/.claude/commands/review-pr.md      →   /review-pr
bazaarx/.claude/commands/scaffold-feature.md →  /scaffold-feature $ARGUMENTS
```

`$ARGUMENTS` = whatever you type after the command name.

---

### Command 1 — `/review-pr`

```markdown
```

```
<!-- .claude/commands/review-pr.md -->
---
description: Review staged changes for BazaarX conventions
allowed-tools: Bash, Read
---
Review staged changes: `git diff --staged`

Check for:
1. Business logic in controllers (must be in service layer)
2. Entities returned from controllers (must use DTOs)
3. Missing @Valid on @RequestBody params
4. Prices not stored as paise integers
5. Missing @Transactional on service write methods

Output: ✅ Good · ⚠️ Warning · ❌ Block
```

**You type:**

```
/review-pr
```

**Claude outputs:**

```
Reviewing staged changes...

✅ OrderService.java — @Transactional present on createOrder()
✅ OrderResponse.java — DTO used, no entity exposure
⚠️ ProductController.java line 42 — price set as 299.99 (decimal). Must be 29999
paise.
❌ CartController.java line 78 — business logic found in controller.
   Move discount calculation to CartService.

2 issues found. Fix before merging.
```

---

## Command 2 — `/scaffold-feature`

```markdown
```

```
<!-- .claude/commands/scaffold-feature.md -->
---
description: Scaffold Entity + Repository + Service + Controller + DTOs
allowed-tools: Read, Write
---
Scaffold a complete Spring Boot feature for: $ARGUMENTS

Follow BazaarX conventions:
1. entity/$ARGUMENTS.java          — @Entity @Data @Builder
2. repository/$ARGUMENTSRepository — extends JpaRepository
3. dto/$ARGUMENTSRequest.java      — Bean Validation
4. dto/$ARGUMENTSResponse.java
5. service/$ARGUMENTSService.java  — @Transactional on writes
6. controller/$ARGUMENTSController — @RestController /api/v1/...

Show the file tree when done.
```

**You type:**

```
/scaffold-feature Wishlist
```

**Claude outputs:**

```
Creating BazaarX Wishlist feature...

backend/src/main/java/com/bazaarx/
├── entity/Wishlist.java            ✅ created
├── repository/WishlistRepository   ✅ created
├── dto/WishlistRequest.java        ✅ created
├── dto/WishlistResponse.java       ✅ created
├── service/WishlistService.java    ✅ created
└── controller/WishlistController   ✅ created

6 files created. All follow BazaarX package and annotation conventions.
```

> 💡 **The wow:** 6 files, correct annotations, correct package structure — from 3 words. This is what repeatable automation feels like.

---

### Command 3 — `/scaffold-react`

```markdown
```

```
<!-- .claude/commands/scaffold-react.md -->
---
description: Scaffold a React component with TypeScript, Tailwind, and React Query h
allowed-tools: Read, Write
---
Scaffold a React component for: $ARGUMENTS

1. frontend/src/components/$ARGUMENTS/$ARGUMENTS.tsx
   — TypeScript props interface, Tailwind, React Query useQuery/useMutation
2. frontend/src/api/$ARGUMENTSApi.ts
   — Axios client matching the Spring Boot endpoints
3. frontend/src/components/$ARGUMENTS/$ARGUMENTS.test.tsx
   — Vitest + React Testing Library skeleton

Prices in paise from API — divide by 100 for display.
```

**You type:**

```
/scaffold-react WishlistCard
```

**Claude outputs:**

```
Creating WishlistCard component...

frontend/src/
├── components/WishlistCard/WishlistCard.tsx     ✅ created
├── components/WishlistCard/WishlistCard.test.tsx ✅ created
└── api/wishlistApi.ts                           ✅ created

Prices fetched in paise, displayed as ₹{(price_paise / 100).toFixed(2)}.
```

---

### Where commands live

```
bazaarx/.claude/commands/    project-level  — committed, shared with team
~/.claude/commands/          personal       — not committed, your own shortcuts
```

---

---

## 4 · Skills — Context That Loads Itself · 15 min

**The problem:** Commands need you to type `/something`. But what about domain knowledge

Claude should just *know* when working on a topic?

**The fix:** A `SKILL.md` whose description is a trigger condition. Claude reads it automatically when context matches.

---

### Commands vs Skills

| | Commands | Skills |
|---|---|---|
| Triggered by | You type `/command` | Claude detects the topic |
| Best for | Repeatable workflows | Domain knowledge, patterns |
| File structure | Single `.md` | Directory + `SKILL.md` |

---

### Skill 1 — Stripe Payments

```markdown
<!-- .claude/skills/stripe-payments/SKILL.md -->
---
name: stripe-payments
description: Load when working with Stripe payments, webhooks, checkout, or refunds
---

## BazaarX Stripe Setup
- Checkout: PaymentController → PaymentService
- Webhooks: StripeWebhookController — verify signature before processing
- Keys: ${stripe.secret-key}, ${stripe.webhook-secret} in application.properties
- Amounts: always paise (INR smallest unit)
- After payment: OrderService.markAsPaid(orderId) — @Transactional, updates status +

## Rules
- Never log full PaymentIntent object — contains sensitive card data
- Webhook endpoint must return 200 fast — offload processing to @Async
- Idempotency key = orderId
```

### You type — without mentioning Stripe:

```
You: Add a refund endpoint for cancelled orders
```

### What Claude does:

```
[Auto-loading skill: stripe-payments — refund topic detected]

Creating refund endpoint for BazaarX...

Following BazaarX Stripe patterns:
- Idempotency key: orderId + "-refund"
- Calling stripe.refunds.create() with paise amount
- Updating order status to CANCELLED via OrderService (existing @Transactional
method)
- Not logging PaymentIntent per BazaarX security rules
```

> 💡 **The wow:** Claude loaded domain-specific rules automatically. You wrote nothing extra. This is the difference between a raw AI and a trained team member.

---

### Skill 2 — Spring JPA Patterns

```markdown
<!-- .claude/skills/spring-jpa/SKILL.md -->
---
name: spring-jpa
description: Load when writing JPA queries, repositories, or working with entity rel
---

## BazaarX JPA Patterns
- Pagination: always use Pageable — never findAll() on products or orders
- N+1 prevention: @EntityGraph or JOIN FETCH for @OneToMany
- Bulk writes: saveAll() — never save() in a loop
- Custom queries: @Query with JPQL preferred over native SQL
- Transactions: @Transactional on service write methods only

## Key indexes
- products(category_id), products(stock_qty)
- orders(user_id, status), orders(created_at)
```

---

### The token saving explained

```
Without Skill: Claude calls PostgreSQL MCP, loads full schema → ~12,000 tokens
With Skill:    Claude reads spring-jpa/SKILL.md              →    ~900 tokens
                                                                 93% saved
```

Skills load only when relevant. CLAUDE.md loads always.
That distinction drives most of your cost savings.

---

---

## 5 · Hooks — Automation That Never Forgets · 15 min

**The problem:** Claude is non-deterministic. You want some things to *always* happen — block a dangerous command, run tests after every write, get notified when Claude needs input.

**The fix:** Shell scripts tied to Claude's event lifecycle. Always run. Cannot be overridden by prompts.

---

### Hook types

| Hook | Fires | Can block? |
|---|---|---|
| `PreToolUse` | Before any tool call | ✅ Yes — exit 2 |
| `PostToolUse` | After tool call completes | No |
| `Notification` | When Claude needs your input | No |
| `PostCompact` | After /compact runs | No |

### Exit codes for PreToolUse:

| Code | Meaning |
|---|---|
| `0` | Allow |
| `1` | Allow, but show warning to Claude |
| `2` | **Block** — stderr becomes Claude's error message |

---

### Hook 1 — Block dangerous shell commands

Add to `.claude/settings.json`:

```
json
```

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{ "type": "command", "command": "scripts/guard.sh" }]
    }]
  }
}
```

```bash
#!/bin/bash
# scripts/guard.sh
CMD=$(cat | jq -r '.tool_input.command')

if echo "$CMD" | grep -qiE 'rm -rf|DROP TABLE|TRUNCATE'; then
  echo "BLOCKED: Destructive command not allowed. Use a migration or ask the team."
  exit 2
fi
exit 0
```

**What the trainer sees when Claude tries a blocked command:**

```
Claude: I'll clean up the build directory...

[Hook: guard.sh]
BLOCKED: Destructive command not allowed. Use a migration or ask the team.

Claude: I can't run that command — it's blocked by the project's guard hook.
        I'll suggest an alternative approach instead.
```

The hook ran. The command never executed. Claude adapted its response.

---

### Hook 2 — Run tests after every file write

```json
```

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write",
      "hooks": [{ "type": "command", "command": "scripts/test-on-write.sh" }]
    }]
  }
}
```

```bash
#!/bin/bash
# scripts/test-on-write.sh
FILE=$(cat | jq -r '.tool_input.path')

if echo "$FILE" | grep -q 'backend/src/main/java'; then
  cd backend && mvn test -q 2>&1 | grep -E 'FAIL|ERROR|BUILD' | head -5
fi

if echo "$FILE" | grep -q 'frontend/src'; then
  cd frontend && npx vitest run --reporter=dot 2>&1 | tail -3
fi
```

**What it looks like after Claude writes a Java file:**

```
Claude: Created OrderService.java with createOrder() method.

[Hook: test-on-write.sh — running affected tests]
Tests run: 12, Failures: 0, Errors: 0 — BUILD SUCCESS

Claude: Tests pass. Moving to the next file.
```

Every write auto-validates. Failures surface before Claude moves on.

---

### Hook 3 — Desktop notification when Claude waits

```bash
#!/bin/bash
# scripts/notify.sh  (macOS)
MSG=$(cat | jq -r '.message')
osascript -e "display notification \"$MSG\" with title \"BazaarX · Claude\""
```

```json
```

```
{
  "hooks": {
    "Notification": [{
      "matcher": "",
      "hooks": [{ "type": "command", "command": "scripts/notify.sh" }]
    }]
  }
}
```

Now you can switch tabs or make tea. Claude pings you when it needs input.

---

### Manage hooks without editing JSON

```
/hooks
```

Opens an interactive UI to create, edit, and toggle hooks.

---

---

## 6 · MCP Servers — Real Tools, Real Data · 10 min

**The problem:** Without MCP, Claude can only read files and run bash. It can't query your live database, create GitHub issues, or call APIs — unless you paste everything manually.

**The fix:** Connect external services once. Claude calls them in natural language.

---

### Connect BazaarX tools

```bash
# GitHub — issues, PRs, code review
claude mcp add --transport http github https://mcp.github.com/mcp \
  --header "Authorization: Bearer $GITHUB_TOKEN"

# Slack — team notifications
claude mcp add --transport http slack https://mcp.slack.com/mcp \
  --header "Authorization: Bearer $SLACK_TOKEN"

# List what's connected
claude mcp list
```

---

### What you can do with natural language

```
You: Create a GitHub issue — "Checkout fails when cart has more than 10 items"
You: Post to the #bazaarx-releases Slack channel: "v1.4 deployed to staging"
You: List all open PRs assigned to me
```

### Expected output for the GitHub issue:

```
Claude: Creating GitHub issue...

Issue #247 created:
  Title: Checkout fails when cart has more than 10 items
  URL:   https://github.com/your-org/bazaarx/issues/247
  Label: bug (auto-assigned)
```

No browser. No context switching. Stay in the terminal.

---

### MCP vs Skill — pick the right tool

| Need | Use |
|---|---|
| Live data or API calls | MCP |
| Patterns and conventions | Skill |
| MCP docs that rarely change | Convert docs → Skill (big token saving) |

---

### Manage MCP connections

```bash
claude mcp list            # see all connected servers
claude mcp remove slack    # disconnect
claude mcp test github     # verify connection is working
```

---

---

## 7 · Subagents — Delegate Without Losing Context · 15 min

**The problem:** Complex tasks bloat the main session. Reading 200 Java files to find a pattern stays in your context window permanently — slowing every subsequent response and inflating

cost.

**The fix:** Delegate to a subagent. It has its own isolated context. It returns only a summary to you.

---

### The architecture

```
Main session (you talk here)
    │
    ├── spawns ──▶ Subagent A   own context · own tools · own model
    ├── spawns ──▶ Subagent B   runs in parallel
    │
    └── receives summaries only — not the full work log
```

---

### Create `.claude/agents/code-reviewer.md`

```markdown
---
name: code-reviewer
description: Reviews BazaarX code for conventions, security, and JPA issues. Invoke
model: claude-haiku-4-5
tools: Read, Bash
---

You are a senior Java/React engineer reviewing BazaarX changes.

For every review:
1. git diff --staged
2. Check CLAUDE.md conventions
3. Flag: business logic in controllers, entities in responses, missing @Transactiona
4. Flag: N+1 risks — repository calls inside loops
5. Flag: React components fetching directly instead of using src/api/

Output: ✅ Good · ⚠️ Warning · ❌ Block. One line per finding.
```

---

### Invoke it

```
@code-reviewer
```

**Expected output:**

```
[Spawning subagent: code-reviewer (claude-haiku-4-5)]
[Subagent context is isolated from main session]

code-reviewer: Reviewing staged changes...

✅ WishlistService.java    — @Transactional on addItem(), correct
✅ WishlistResponse.java   — DTO used, no entity fields exposed
⚠️ WishlistController.java line 31 — calls wishlistRepo directly.
   Move to WishlistService per BazaarX conventions.
❌ WishlistCard.tsx line 18 — fetch('/api/v1/wishlist') called directly.
   Must use src/api/wishlistApi.ts

2 issues. Main session context unchanged.
```

A cheaper model (`haiku`) did the review. The main session stayed clean.

---

### Parallel subagents — the real power

```
You: Audit the checkout flow for security issues, N+1 queries,
     and missing test coverage — run all three in parallel.
```

**Expected output:**

```
[Spawning 3 subagents in parallel]
  → security-auditor   (claude-haiku-4-5)
  → n1-detector        (claude-haiku-4-5)
  → test-coverage-scan (claude-haiku-4-5)

[All 3 complete — merging findings]

Security:      ⚠️ StripeWebhookController missing signature check on /refund
N+1 queries:   ❌ OrderService.getOrdersForUser() calls productRepo inside loop
(line 67)
Test coverage: ⚠️ CheckoutService.applyDiscount() has no test

3 findings across 3 dimensions. Elapsed: 18 seconds.
```

Without subagents: sequential, 3× slower, all 3 analyses in main context permanently.
With subagents: parallel, only the summary comes back.

---

### Background tasks

```
Ctrl+B    Send a subagent to background. Keep typing in main session.
```

---

---

## 8 · Agents & Agent Teams — Full Orchestration · 10 min

**The main agent is the orchestrator.** It reads CLAUDE.md, auto-loads Skills, enforces Hooks, calls MCP tools, and spawns Subagents. You don't configure it — you understand it.

---

### One prompt — all layers working together

```
You: Flash sale tomorrow. 20% off Electronics.
     Write the social copy for the top 5 products,
     create a GitHub issue for the rollback plan,
     and notify the team on Slack.
```

### What happens under the hood:

```
Main Agent (orchestrator)
  |
  ├── spawn copywriter subagent (parallel)
  |     → generates 5 social posts based on top Electronics products
  |
  ├── GitHub MCP
  |     → creates issue "Flash Sale Rollback Plan — Electronics 20% off"
  |
  ├── Slack MCP
  |     → posts to #bazaarx-ops
  |
  ├── PreToolUse hook
  |     → validates every Bash command before execution
  |
  └── Skills auto-loaded
        stripe-payments (Stripe context), spring-jpa (JPA context)
```

Everything you built in this training — working together from one prompt.

---

### The full picture

```
┌──────────────────────────────────────────────────────────┐
|  CLAUDE.md     What Claude knows (always loaded)         |
|  settings.json What Claude can do (team guardrails)      |
├──────────────┬──────────────────────────────────────────┤
|  Commands    |  /review-pr  /scaffold-feature           |
|  Skills      |  stripe-payments  spring-jpa  (auto)     |
|  Hooks       |  guard  test-on-write  notify            |
├──────────────┼──────────────────────────────────────────┤
|  MCP         |  GitHub  Slack                           |
|  Subagents   |  code-reviewer  n1-detector              |
|  Agent Teams |  peers — cross-check each other          |
├──────────────┴──────────────────────────────────────────┤
|  Plugins     Bundle + share all of the above             |
└──────────────────────────────────────────────────────────┘
```

---

### Agent Teams — when subagents aren't enough

Subagents report to the main agent (boss → worker). Agent Teams are **peers** — they talk to each other, cross-check, self-assign.

```
Shift+T   ← Open Agent Teams panel

You: Create a team — Security Analyst, Performance Engineer, Test Auditor.
     Audit the checkout service. Cross-check each other's findings.
```

| | Subagents | Agent Teams |
|---|---|---|
| Relationship | Boss → worker | Peers |
| Talk to each other | No | Yes |
| Cost | Moderate | High |
| Use when | Parallel independent tasks | Cross-validation, audits |

Use Agent Teams sparingly — security audits, architecture reviews, pre-launch checks.

---

---

## 9 · Token Optimization — Fast and Affordable · 10 min

**Tokens = time + money.** Five patterns cover 90% of cost on BazaarX.

---

### Pattern 1 — Right model for the task

```
claude-haiku-4-5   → review, search, linting, mechanical tasks  (cheapest)
claude-sonnet-4-6  → implementation, reasoning                  (default)
claude-opus-4-6    → architecture, genuinely hard problems       (most expensive)
```

Set in subagent frontmatter:

```yaml
model: claude-haiku-4-5
```

Switch mid-session:

```
/model haiku
```

---

### Pattern 2 — MCP docs → Skill (biggest single saving)

```
GitHub MCP loads full API surface on every call  →  ~10,000 tokens
BazaarX github-workflow Skill (just what we use) →    ~600 tokens
                                                      94% saved
```

Write a Skill with only what your project actually uses.

---

### Pattern 3 — Compact before switching tasks

```
/compact Focus on the payment service and current test failures
```

Use at ~70% context. You keep the intent, drop the verbosity. Cost drops immediately.

---

### Pattern 4 — Subagents for exploration

```
❌ Expensive: ask main Claude to search 200 Java files
   → all 200 files permanently in main context
```

```
✅ Cheap: spawn a subagent for the search
   → only the 3-line summary enters main context
```

---

### Pattern 5 — CLAUDE.md hygiene

Every line loads on every session start. Quarterly audit:

- Remove resolved TODOs
- Move deep docs into Skills (load only when needed)
- Bullet points, not paragraphs

---

### Cost impact on a typical BazaarX session

| Approach | ~Tokens | ~Cost |
|---|---|---|
| No optimization | 120,000 | ₹180 |
| Skills + model routing | 35,000 | ₹52 |
| + Subagents for exploration | 18,000 | ₹27 |
| **Total saving** | **85%** | **₹153** |

---

---

## 10 · Plugins — Share Your Tooling · 5 min

**The problem:** You've built great Commands, Skills, Subagents, and Hooks for BazaarX. Another team starts a similar project and rebuilds everything from scratch.

**The fix:** Bundle into a Plugin. Share via GitHub. Install with one command.

---

### Plugin structure

```
bazaarx-plugin/
├── claude-plugin.json       manifest
├── skills/
|   ├── stripe-payments/
|   └── spring-jpa/
├── agents/
```

```
│   └── code-reviewer.md
└── commands/
    ├── review-pr.md
    └── scaffold-feature.md
```

---

### Install a plugin

```bash
/plugin add github.com/your-org/bazaarx-plugin

/plugin add marketplace   # browse Anthropic's official plugins
```

All Skills, Agents, and Commands from the plugin are immediately available.

---

---

## Quick Reference

### Daily session commands

| Command | Does |
|---|---|
| `/compact` | Summarize history, free context — use at 70%+ |
| `/clear` | Wipe and start fresh |
| `/model haiku` | Switch to cheaper model |
| `/rename` | Name this session |
| `/resume` | Pick up a named session |
| `/diff` | See pending changes before committing |
| `/hooks` | Manage hooks interactively |

### In-session shortcuts

| Shortcut | Does |
|---|---|
| `!git diff` | Run shell directly — no conversational overhead |
| `@src/service/OrderService.java` | Reference a file inline |

| Shortcut | Does |
|---|---|
| `Ctrl+B` | Send task to background, keep typing |
| `Esc Esc` | Rewind — roll back code only, or conversation only |
| `Alt+P` | Switch model without losing your typed prompt |

### CLI flags

```bash
claude -c                            # continue most recent session
claude -r "bazaarx-payment-fix"      # resume by name
claude -n "wishlist-feature"         # start session with a name
claude -p "..." --output-format json # non-interactive, for CI scripts
```

---

---

## 3 Questions for Every Task

**1. CLAUDE.md or Skill?** Needed every session → CLAUDE.md · Needed only for a topic → Skill

**2. Main agent or subagent?** Exploratory / mechanical / parallel → Subagent

**3. Which model?** Sonnet by default · Haiku for subagents · Opus for hard problems only

---

## Take-Home Exercises

Each exercise takes 15–20 minutes on your own laptop.

| # | Exercise |
|---|---|
| 1 | Write a `CLAUDE.md` for one of your current projects |
| 2 | Create one `/command` for a workflow your team repeats weekly |
| 3 | Write one Skill for your most complex domain area (payments, auth, etc.) |
| 4 | Add one `PreToolUse` hook that protects your most critical resource |
| 5 | Create one `code-reviewer` subagent tailored to your team's conventions |

---

## BazaarX File Reference

```
bazaarx/
├── CLAUDE.md                          ← Section 1
├── .claude/
|   ├── settings.json                  ← Section 2
|   ├── commands/
|   |   ├── review-pr.md               ← Section 3
|   |   ├── scaffold-feature.md        ← Section 3
|   |   └── scaffold-react.md          ← Section 3
|   ├── skills/
|   |   ├── stripe-payments/SKILL.md   ← Section 4
|   |   └── spring-jpa/SKILL.md        ← Section 4
|   └── agents/
|       └── code-reviewer.md           ← Section 7
└── scripts/
    ├── guard.sh                       ← Section 5
    ├── test-on-write.sh               ← Section 5
    └── notify.sh                      ← Section 5
```

---